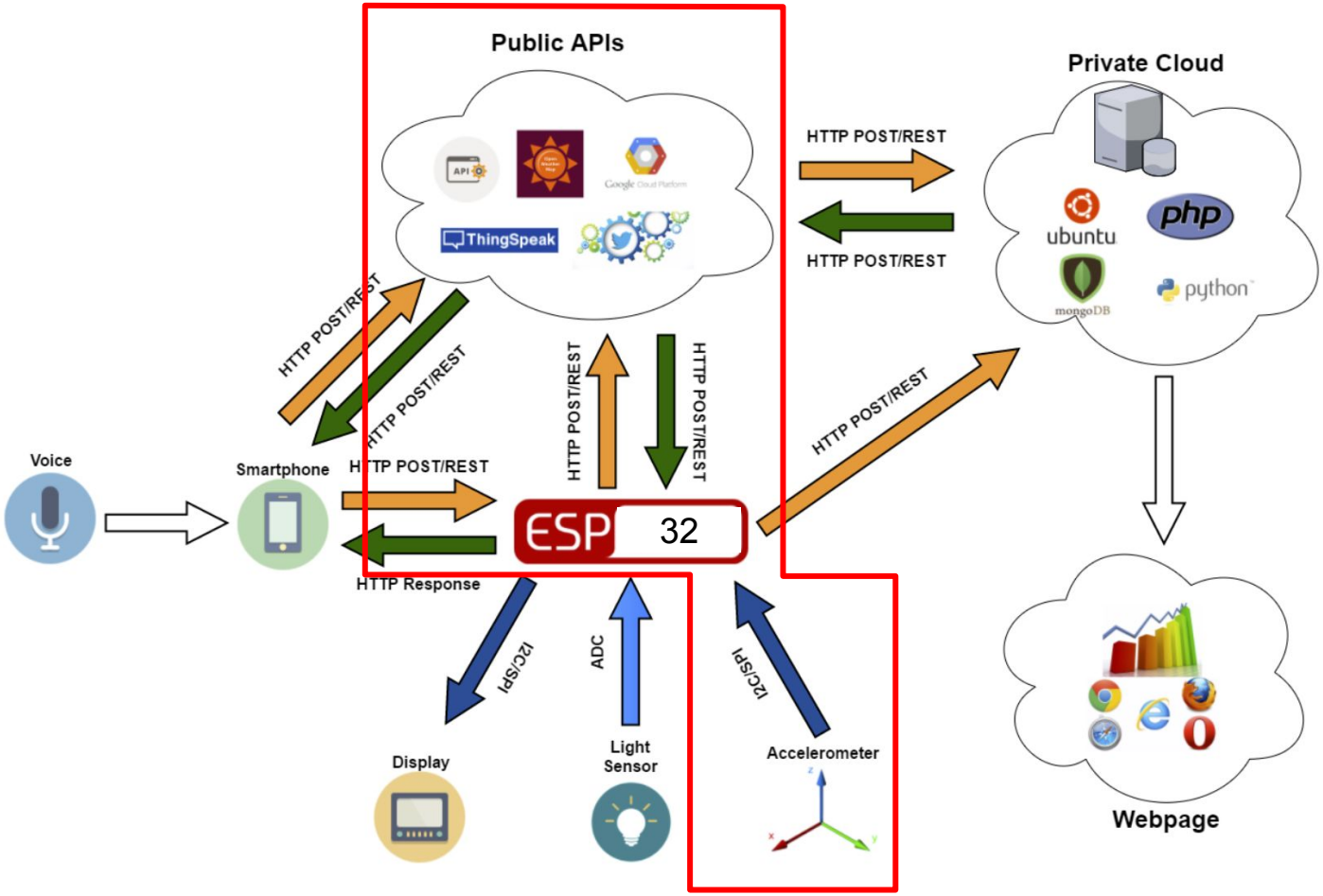
AIoT - Intelligent and Connected Systems
EECS E4764 Fall 26

Lab 4

Lab Overview



Part 1

Connecting to the Accelerometer

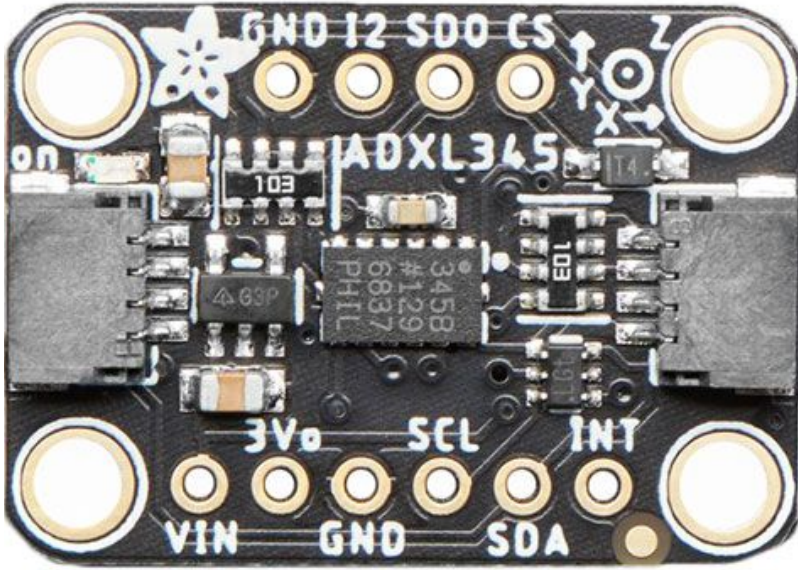
Connecting to Accelerometer (ADXL345)

In this lab, you need to interface with the accelerometer using **SPI**.

In this lab, VIN, GND, SDA(MOSI/SDI), SDO(MISO/SDO), CS, SCL(SCK) need to be connected to ESP.

The datasheet has everything you need:

https://www.sparkfun.com/datasheets/Sensors/Accelerometer/ADXL345.pdf



Connecting to Accelerometer (ADXL345)

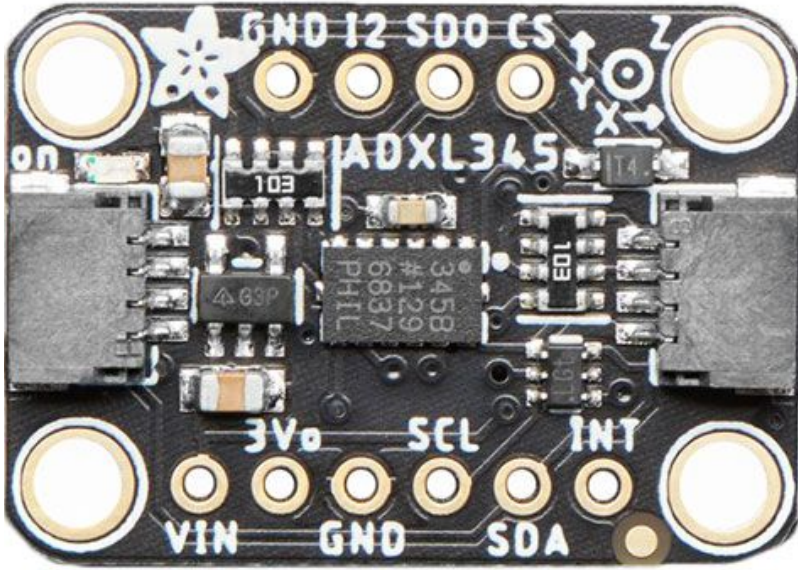
SPI stands for **Serial Peripheral Interface**. It's a way for microcontrollers (like your ESP32) to talk to other devices (like sensors, displays, or memory chips).

Think of it like a **walkie-talkie system** between two devices:

- One device is the **master** (your ESP32) → controls the conversation.
- The other device is the **slave** (like ADXL345 accelerometer).

It uses **SCK (Clock)** to sync data, **MOSI** to send data from master to slave, and **MISO** to receive data from slave to master.

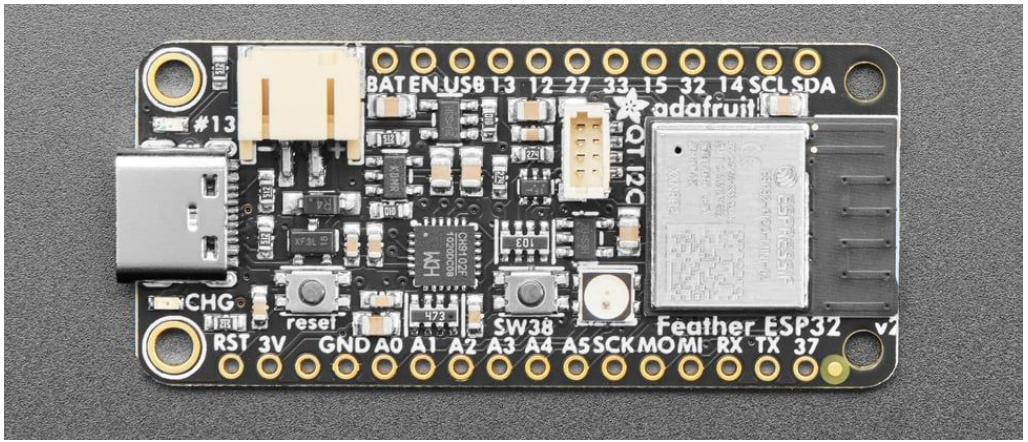
The **CS (Chip Select)** wire tells the device when the ESP32 wants to communicate.

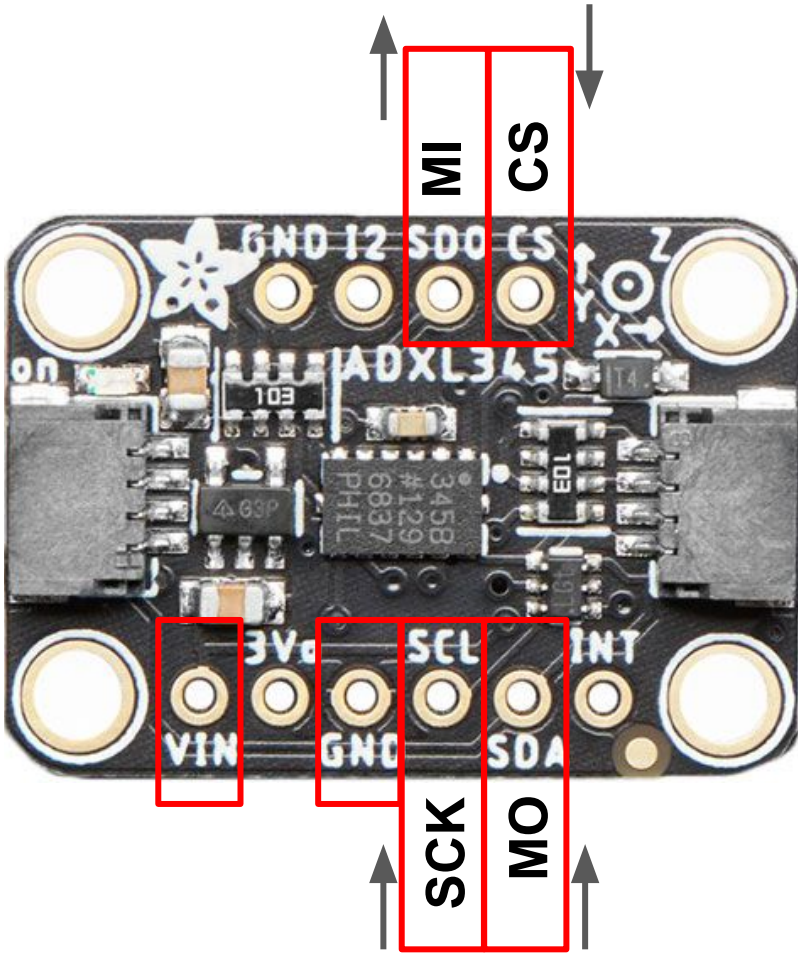


Connecting to Accelerometer (ADXL345)

Choose your own GPIO for CS

(e.g. 15)





Initialization of ADXL345

Before you initialize the sensor, you need to know how to write data to the sensor. Details in the datasheet page 9 (SPI 4-wire)

The registers you need to initialize are **0x31, 0x2d, 0x2c, 0x2e, 0x38**. Please read the register map at page 14 and find out what value you should write in.

SPI Read/Write on ADXL345

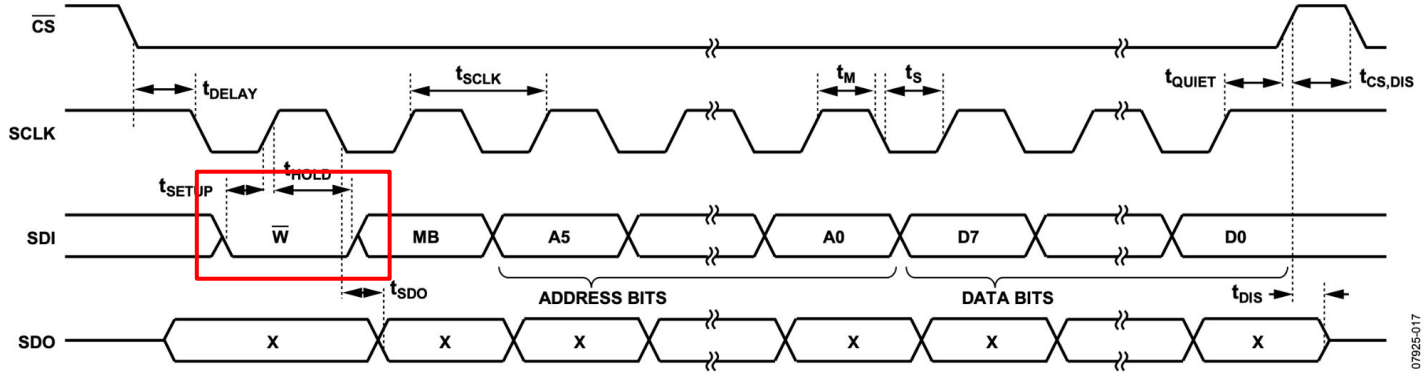


Figure 5. SPI 4-Wire Write

---

1. SPI Write (Top Diagram)
CS: Chip Select
Goes low → ADXL345 knows the master (ESP32) is starting communication.
Goes high → communication ends.

SCLK: Clock signal
Tells the sensor when to read the next bit.
Every rising/falling edge, the device reads or writes a bit.

SDI (MOSI): Master Out → Slave In
Carries register address + data from ESP32 to ADXL345.
The first bit indicates W (Write) or R (Read).
W = 0 → write to register
R = 1 → read from register
Next bits: A5–A0 → register address, D7–D0 → data.

SDO (MISO): Slave Out → Master In
Not used during write (X = ignored)
During read, sends data from sensor to ESP32.

SPI Read/Write on ADXL345

Very similar to write, but:

- First bit of SDI = **R** → read mode
- After sending register address, the sensor outputs **data on SDO (MISO)**
- Data is read by ESP32 bit by bit, synchronized with SCLK

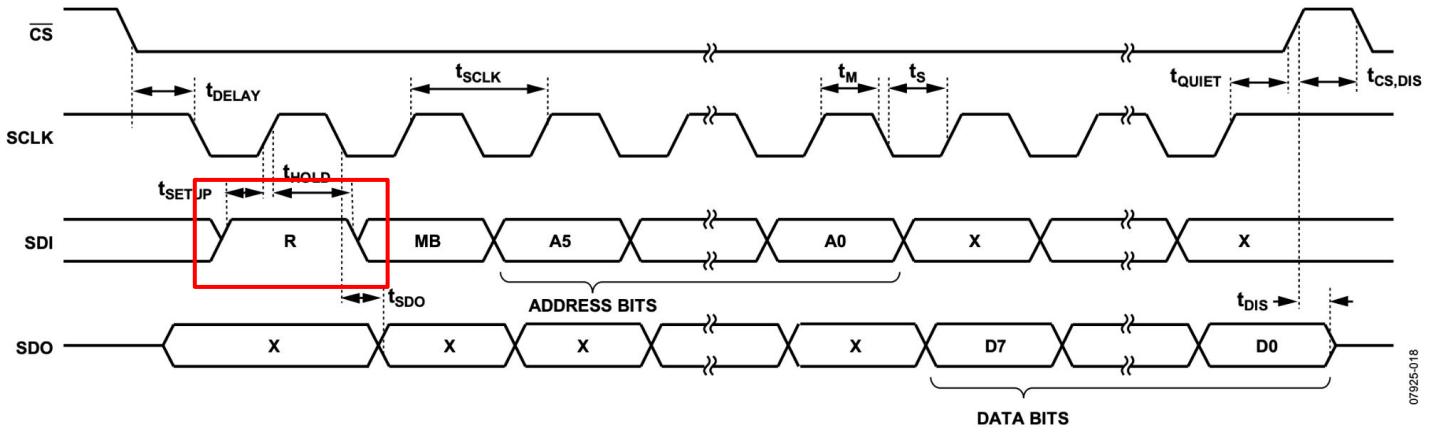


Figure 6. SPI 4-Wire Read

Reading accelerometer data

```
# Concatenate data (combine 2 bytes into 1 number)

x = buffer[2] << 8 | buffer[1]

# Convert to 2s complement

if x > 32767:

   x -= 65536
```

The ADXL345 accelerometer outputs raw measurement data for the **X, Y, and Z axes** in registers `0x32–0x37`, with each axis using two consecutive 8-bit registers: `DATAx0` holds the **least significant byte (LSB)** and `DATAx1` holds the **most significant byte (MSB)**. Together they form a **16-bit signed integer** that represents the acceleration along that axis.

To reconstruct the value, the two bytes must be concatenated as `(DATAx1 << 8) | DATAx0`. Because the accelerometer uses **two's complement format**, the number can represent both positive and negative acceleration. If the combined value is larger than 32767 (the maximum positive 16-bit value), it should be adjusted by subtracting 65536 to obtain the correct negative result.

This signed value directly corresponds to acceleration in units of *g*, depending on the sensitivity setting in the **DATA_FORMAT register (0x31)**. Reading all six bytes in one transaction is recommended to prevent mismatches between high and low bytes.

SPI Read/Write on ADXL345

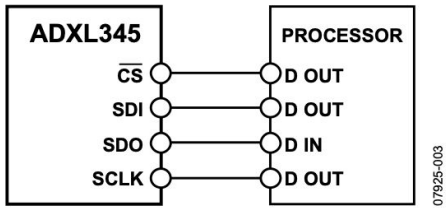


Figure 4. 4-Wire SPI Connection Diagram

To read or write multiple bytes in a single transmission, the multiple-byte bit, located after the R/$\overline{W}$ bit in the first byte transfer (MB in Figure 5 to Figure 7), must be set. After the register addressing and the first byte of data, each subsequent set of clock pulses (eight clock pulses) causes the ADXL345 to point to the next register for a read or write. This shifting continues until the clock pulses cease and $\overline{CS}$ is deasserted. To perform reads or writes on different, nonsequential registers, $\overline{CS}$ must be deasserted between transmissions and the new register must be addressed separately.

Reading from ADXL345

Details in datasheet page 9.

Be careful that the first bit (read/write) need to be set to **1**, in order to enter reading mode.

You can use **SPI.readinto** function

https://docs.micropython.org/en/latest/library/machine.SPI.html#machine.SPI.readinto

Starter Code

```
from machine import Pin, SPI
import time

# === SPI Setup ===
# Try the common HSPI pins on Feather Huzzah ESP32
spi = SPI(1, baudrate=5000000, polarity=1, phase=1,
          sck=Pin(14), mosi=Pin(13), miso=Pin(12))
cs = Pin(15, Pin.OUT)
cs.value(1)  # Deselect device by default

# SPI Commands
SPI_READ = 1 << 7
SPI_SINGLE_BYTE = 0 << 6

# === Test Device ID ===
def read_device_id():
    cs.value(0)
    spi.write(bytearray([SPI_READ | SPI_SINGLE_BYTE | 0x00]))  # DEVID register = 0x00
    buf = bytearray(1)
    spi.readinto(buf)
    cs.value(1)
    return buf[0]

# Main
device_id = read_device_id()
print(f"[TEST] Device ID read: 0x{device_id:02X}")

if device_id == 0xE5:
    print("[TEST] ADXL345 detected successfully!")
else:
    print("[TEST] ADXL345 not detected. Check wiring and SPI pins!")
```

Or SCK - pin 5. MO - pin 19. MI - pin 21. Depending on the SPI bus you use.

You should see 0xE5 in the result

Starter Code

```python
from machine import Pin, SPI
import time

# === SPI Setup ===
# Try the common HSPI pins on Feather Huzzah ESP32
spi = SPI(
    1, baudrate=5000000, polarity=1, phase=1, sck=Pin(5), mosi=Pin(19), miso=Pin(21)
)
cs = Pin(15, Pin.OUT)
cs.value(1)  # Deselect device by default

# SPI Commands
SPI_READ = 1 << 7
SPI_SINGLE_BYTE = 0 << 6


# === Test Device ID ===
def read_device_id():
    cs.value(0)
    spi.write(bytearray([SPI_READ | SPI_SINGLE_BYTE | 0x00]))  # DEVID register = 0x00
    buf = bytearray(1)
    spi.readinto(buf)
    cs.value(1)
    return buf[0]


# Main
device_id = read_device_id()
print(f"[TEST] Device ID read: 0x{device_id:02X}")

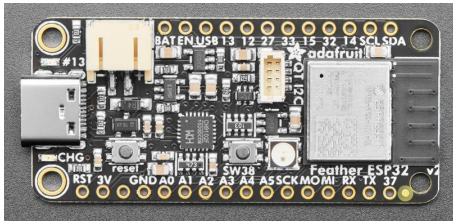
if device_id == 0xE5:
    print("[TEST] ADXL345 detected successfully!")
else:
    print("[TEST] ADXL345 not detected. Check wiring and SPI pins!")
```

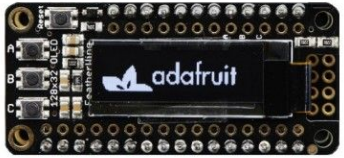
You should see 0xE5 in the result

Part 2

Connecting to the Internet and using APIs

Overview





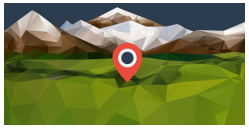
Request

Geolocation information

Geolocation information

Weather

Request



OpenWeather

twitter

What do you need to do?

- Connect the Huzzah to the Internet. You will need to know the WiFi name and password to access the network.
- Once you are connected to the internet, you need to make a REST call to fetch your location information. Feel free to use any of the available APIs which can give you this information. Store the location information for future reference in your code. Print your latitude and longitude on the OLED.
- Use your location information to get the weather information from another API. Print your Weather information on the OLED. OLED has limited display space, so printing the temperature in your location with a one word weather summary will work. For example “25 C, Sunny”.
- Post your weather to ntfy.sh.

Note

1. Location information means Latitude and Longitude in the context of this lab exercise.

2. There are many other APIs that allow you to send messages, other than the ntfy.sh API; you are welcome to use any of them. The messages you send do not have to be typed out and can be preset.

3. The same applies to Geolocation and Weather APIs. Feel free to use any API that you are comfortable using.

4. Remember that some APIs might need you to register and/or enter a billing information prior to usage. In most cases, pricing is based on number of concurrent calls and API usage on a monthly basis. Do read the terms carefully before signing up for an API where you have to enter billing information.

Lab 4 Checkpoints

1. On top of the combined system from the last lab, implement text scrolling based on accelerometer readings.
2. Interface with a Geolocation API and display your coordinates on the OLED display.
3. Feed your coordinates into an API to receive information about the weather in your area; display the results on your OLED display.
4. Post a message from the ESP32. An example message could be your location coordinates with the weather in that location. Also, open the website or app with your PC or phone to show the message by subscribing to the topic.